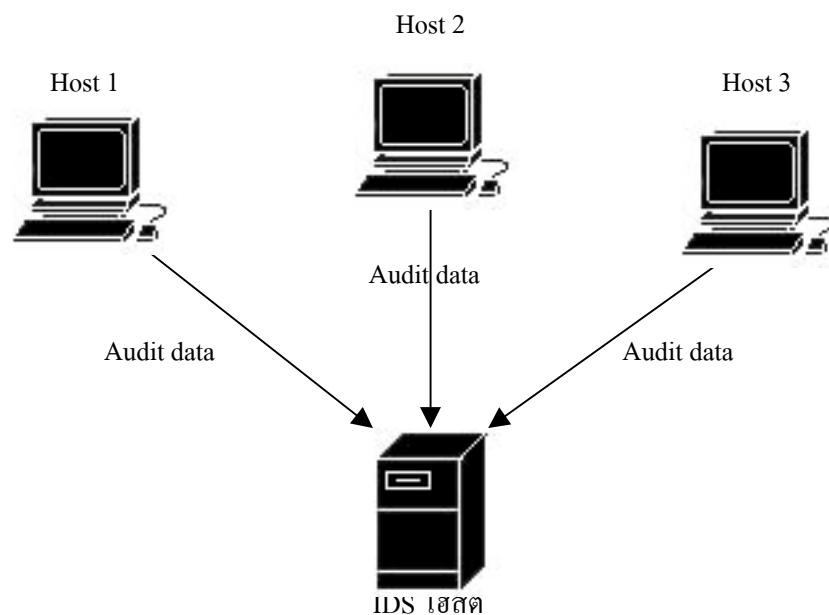


บทที่ 4

การทำงานของระบบ

4.1 ภาพรวมของระบบตรวจจับผู้บุกรุก



รูปที่ 4.1 ภาพรวมการทำงานของระบบตรวจจับผู้บุกรุก

จากรูปที่ 4.1 จะทราบว่าระบบตรวจจับผู้บุกรุกนั้นจะมีศูนย์กลางการทำงานอยู่ที่ IDS โฮสต์โดยแยกออกมาจากโฮสต์อื่นๆ ซึ่งทำหน้าที่วิเคราะห์ข้อมูลที่ส่งมาจากโฮสต์ที่วางตัวตรวจจับข้อมูลพฤติกรรมผู้ใช้ไว้

ข้อดีของการแยก IDS โฮสต์ออกมาจากโฮสต์ตัวอื่น

- เมื่อแยกส่วนที่เป็นการวิเคราะห์ออกมาทำให้โฮสต์ที่วางระบบตรวจจับผู้บุกรุกไว้ทำหน้าที่แค่ส่งข้อมูลพฤติกรรมผู้ใช้งานมาซึ่งจะทำให้โฮสต์ที่วางระบบตรวจจับผู้บุกรุกไว้ถูกใช้ทรัพยากรระบบไปไม่มาก
- สามารถวิเคราะห์พฤติกรรมของผู้ใช้ในหลายๆ โฮสต์ได้พร้อมกัน
- ถ้าเราต้องการวางระบบตรวจจับผู้บุกรุกไว้ในเครื่องที่มีระบบปฏิบัติการอื่นๆ (ในโครงการนี้ได้พัฒนาเฉพาะระบบปฏิบัติการลินุกซ์เท่านั้น) ก็สามารถทำได้ง่ายโดยทำการพัฒนาโปรแกรมเฉพาะส่วนเก็บข้อมูลผู้ใช้ให้ส่งข้อมูลพฤติกรรมผู้เข้ามาให้ IDS โฮสต์ในรูปแบบที่กำหนดไว้เท่านั้น

- ช่วยให้ผู้ใช้และระบบทำงานได้ง่ายขึ้นเพราะทำการตรวจสอบพฤติกรรมผู้ใช้งานหรือความผิดปกติของระบบสามารถทำได้โดยเริ่มทำการตรวจสอบที่เครื่อง IDS โฮสต์ก่อน และเมื่อเจอปัญหาจึงไปจัดการที่โฮสต์นั้นๆ

4.2 โครงสร้างระบบตรวจจับผู้บุกรุกโดยวิธีการตรวจสอบการใช้งานระบบที่ผิดปกติ

ระบบตรวจจับผู้บุกรุกจะแบ่งออกเป็น 3 ส่วนหลักคือ

1. ส่วนเก็บข้อมูลพฤติกรรมผู้ใช้งาน
2. ส่วนเก็บข้อมูลและวิเคราะห์พฤติกรรมผู้ใช้งาน
3. ส่วนตอบสนองการวิเคราะห์ ทำการรายงานผลการวิเคราะห์เมื่อพบพฤติกรรมการใช้งานที่ผิดปกติไป

4.2.1 ส่วนเก็บข้อมูลพฤติกรรมผู้ใช้งาน

ส่วนเก็บข้อมูลพฤติกรรมผู้ใช้งานจะถูกติดตั้งอยู่ที่โฮสต์ที่ต้องการวางระบบตรวจจับผู้บุกรุกไว้ โดยการทำงานก็จะทำการติดตามการใช้งานผ่านทางเคอร์เนลเรียกใช้ซิสเต็มคอลล์ `execve` โดยปกติซิสเต็มคอลล์ตัวนี้จะถูกเรียกใช้งานเมื่อทำการรันไฟล์โปรแกรมต่างๆ ซึ่งจะมีการส่งทั้งคำสั่งและอาร์กิวเมนต์ (argument) ของคำสั่งนั้นๆ เข้ามาที่ซิสเต็มคอลล์จากหลักการทำงานดังกล่าวเราสามารถดักจับข้อมูลการใช้งานของผู้ใช้โดยดูที่ลักษณะของคำสั่งที่ใช้และลำดับการใช้งานของคำสั่งได้ โดยเราจะโหลดโมดูลเข้าไปเพื่อทำหน้าที่ดังกล่าวโดยลักษณะการทำงานของโมดูลจะเป็นไปตาม Flowchart ดังรูปที่ 4.2

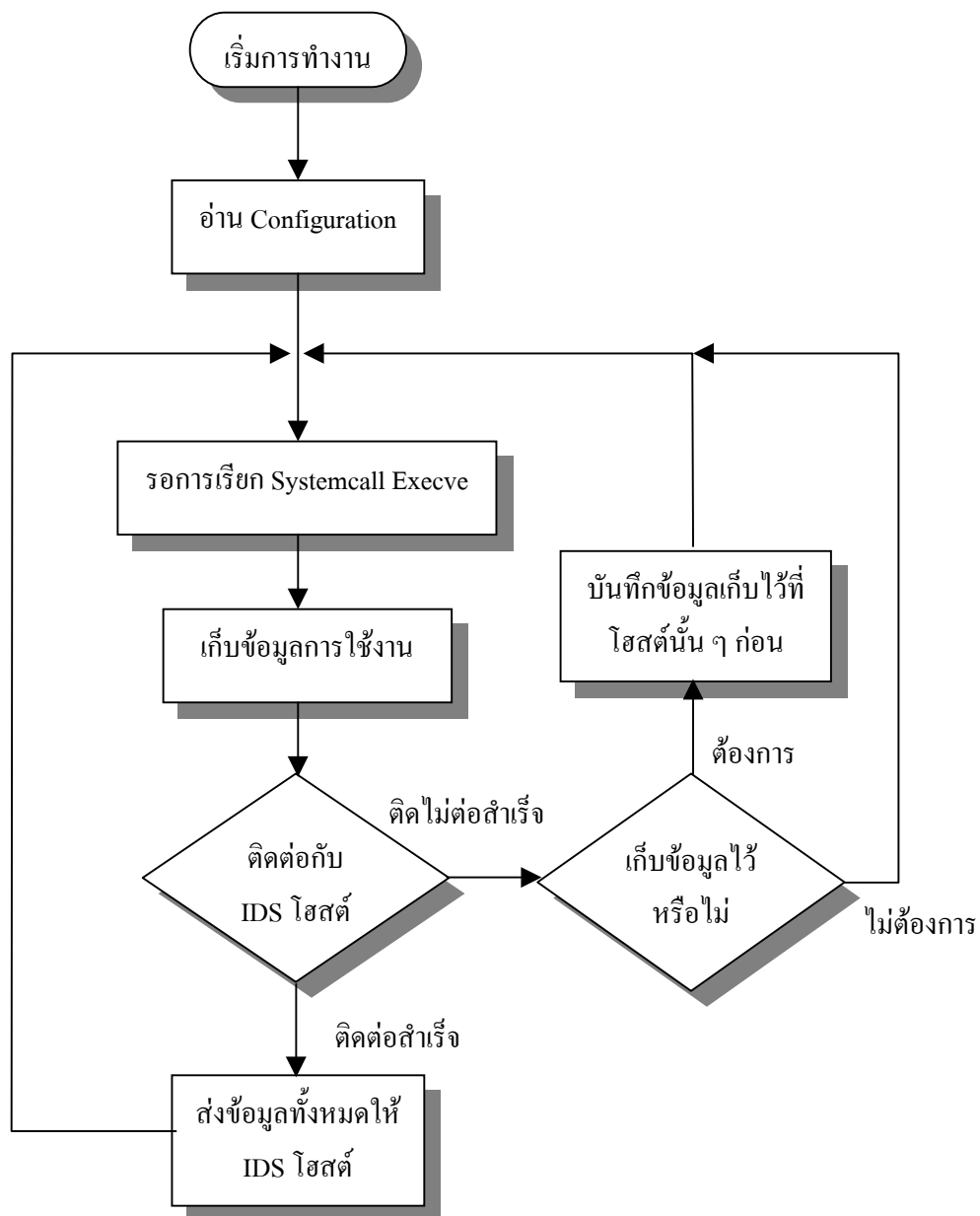
ลักษณะข้อมูลที่ส่งไปให้ IDS โฮสต์จะเก็บเป็นโครงสร้างดังนี้

```
struct execInfor {
    int uid; // หมายเลขประจำตัวผู้ใช้งาน
    char pathname[PATH_LEN]; // ตำแหน่งไฟล์ที่เรียกใช้งาน
    char command[MAX_COMM_LEN]; // คำสั่งและอาร์กิวเมนต์
    unsigned long startTime; // เวลาที่ทำการเรียกคำสั่ง
};
```

หากสังเกตจะพบว่าทั้งๆที่ส่วนของข้อมูลเกี่ยวกับการใช้งานของผู้ใช้นั้นมีมากมายดังได้กล่าวไว้ข้างแล้วในบทที่ 2 แต่เรานำมาใช้เพียงเท่านี้เพราะส่วนวิเคราะห์ของระบบเราต้องการข้อมูลเพียงแค่นี้เท่านั้นถ้าเราเก็บข้อมูลมากไปก็จะทำให้สิ้นเปลืองทรัพยากรระบบและระบบจะทำงานช้าลงเพราะต้องเก็บข้อมูลทุกๆ คำสั่งที่ใช้งานของผู้ใช้ทุกคนที่กำลังใช้งานอยู่และหากสังเกตในอีกแง่หนึ่งก็จะพบว่าสิ่งที่เราทำการเก็บนั้นคล้ายกับข้อมูลที่อยู่ในล็อกไฟล์ `pacct` แต่ความจริงแล้วต่างกันเพราะ

1. การเก็บข้อมูลคำสั่งที่ใช้งานของล็อกไฟล์ `pacct` เก็บเฉพาะคำสั่งที่เรียกใช้งานเท่านั้น ข้อมูลที่เหลือเช่น ออปชัน และอาร์กิวเมนต์ต่าง ๆ จะไม่ถูกเก็บไว้ทำให้ได้ข้อมูลพฤติกรรมการใช้งานที่ไม่ครบถ้วน

2. การเก็บข้อมูลใน packet จะทำขึ้นเมื่อโปรเซสนั้นๆ จบการทำงานแล้วเท่านั้น



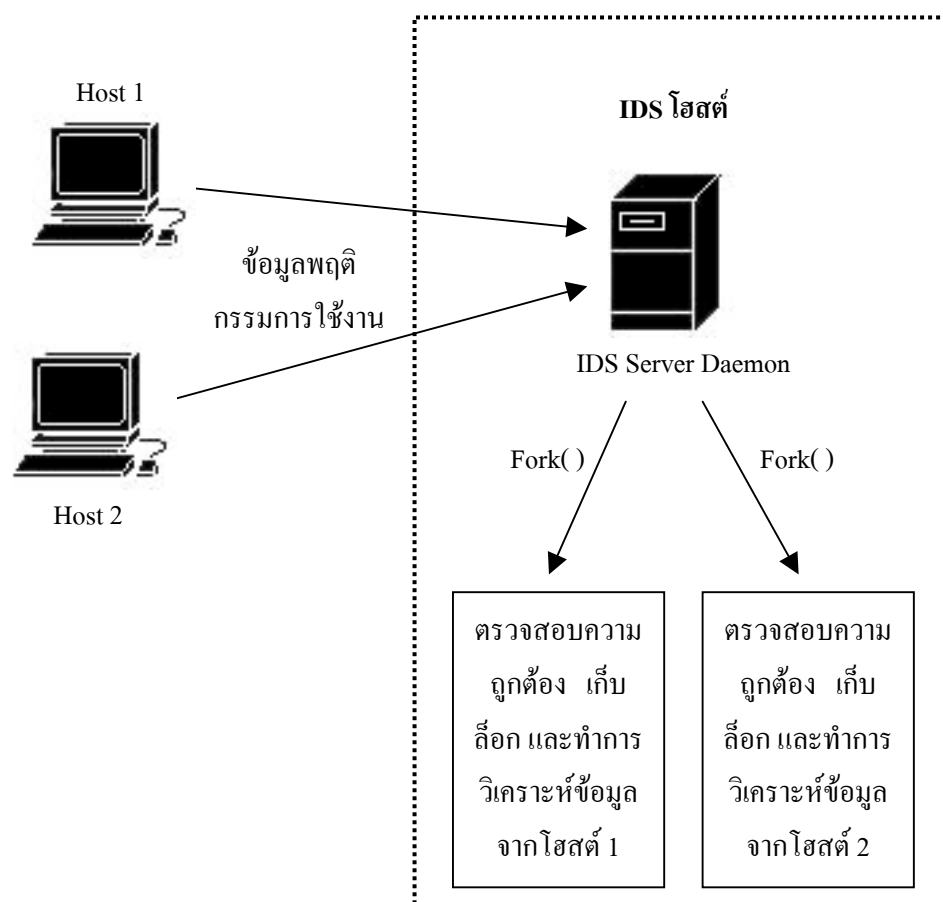
รูปที่ 4.2 ขั้นตอนการทำงานของส่วนเก็บข้อมูลพฤติกรรม

ดังนั้นเพื่อให้ได้ข้อมูลที่ใกล้เคียงกับพฤติกรรมผู้ใช้และมีความถูกต้องมากที่สุดและข้อมูลที่ได้อาจเป็นข้อมูลที่สามารถใช้ในการวิเคราะห์ได้จริงๆ จึงใช้วิธีการเก็บข้อมูลด้วยการติดตามการทำงานของซิสเต็มคอลล์ดังที่กล่าวมา

4.2.2 ส่วนเก็บข้อมูลและวิเคราะห์พฤติกรรมผู้ใช้งาน

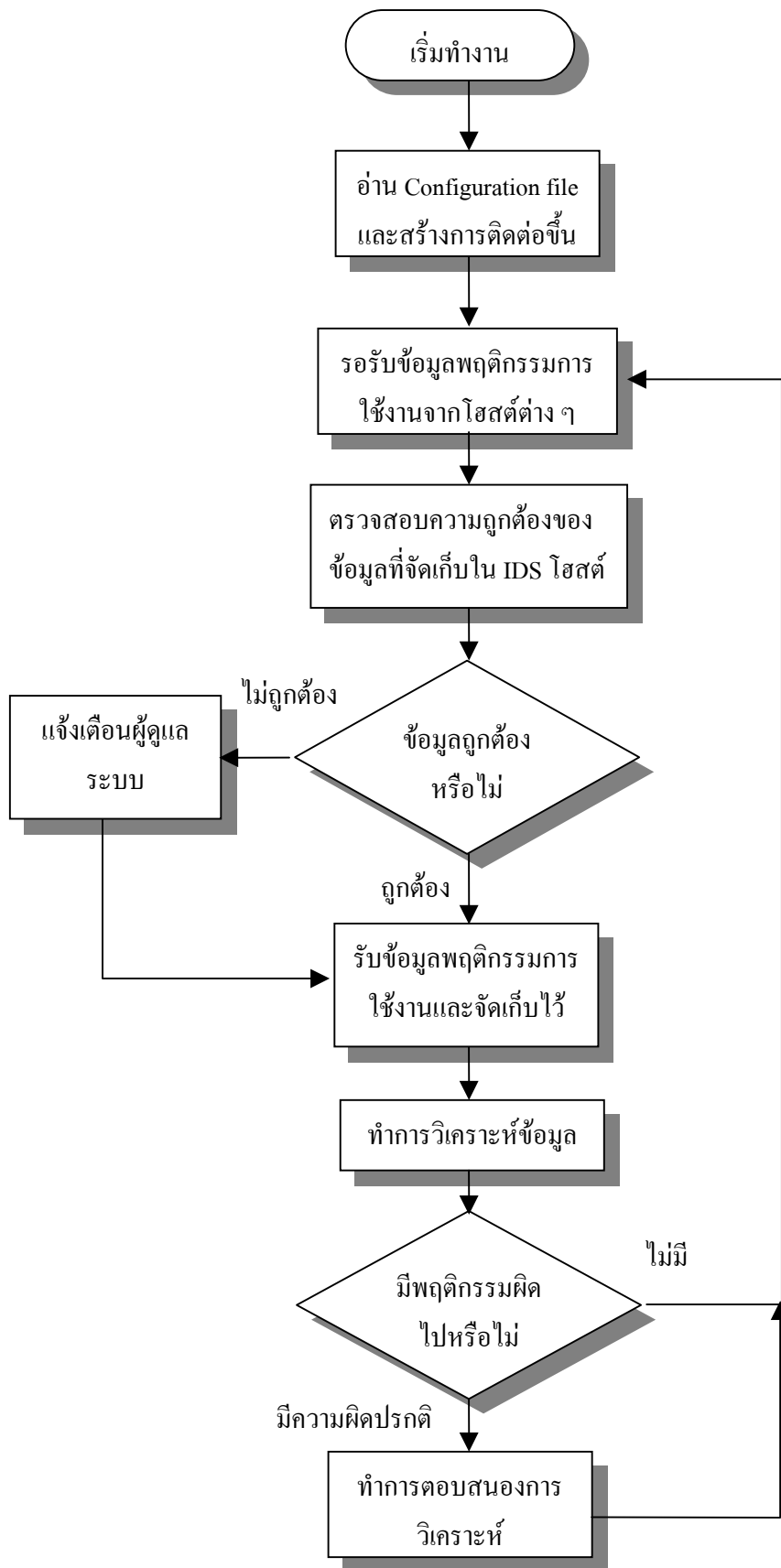
ส่วนเก็บข้อมูลและวิเคราะห์พฤติกรรมผู้ใช้งานจะถูกติดตั้งอยู่ที่ฝั่ง IDS โฮสต์ในส่วนนี้เป็นหน้าที่ของ IDS เดมอน โดยมีหน้าที่ 3 ส่วน คือ

1. ส่วนตรวจสอบความถูกต้องของพฤติกรรมผู้ใช้งาน ทำการตรวจสอบความถูกต้องของข้อมูลที่เก็บใน IDS โฮสต์
2. ส่วนเก็บข้อมูลพฤติกรรมผู้ใช้งาน ทำการรับข้อมูลพฤติกรรมการใช้งานที่ส่งมาจากโฮสต์ต่างๆ และเก็บไว้ใน IDS โฮสต์
3. ส่วนการวิเคราะห์พฤติกรรมผู้ใช้งาน นำข้อมูลในฐานข้อมูลมาผ่านกระบวนการวิเคราะห์และทำการตอบสนองการวิเคราะห์ โดยส่วนการตอบสนองการวิเคราะห์จะขอแยกหัวข้อมออกไป



รูปที่ 4.3 ภาพรวมของส่วนเก็บข้อมูลและวิเคราะห์พฤติกรรมผู้ใช้งาน

การทำงานของ IDS เซิร์ฟเวอร์เดมอนมีลำดับการทำงานดังนี้



รูปที่ 4.4 ขั้นตอนการทำงานของส่วนเก็บข้อมูลและวิเคราะห์พฤติกรรมผู้ใช้งาน

ส่วนตรวจสอบความถูกต้องของข้อมูลพฤติกรรมผู้ใช้งาน

ในส่วนนี้จะทำการสร้าง MD (Message digest) ของฐานข้อมูลแต่ละคนแยกออกเป็นไฟล์ออกมาอีกส่วนหนึ่งโดยแต่ละคนก็จะมี MD คนละ 1 ไฟล์ เหตุผลที่ต้องทำ MD ของแต่ละคนแยกกันคนละไฟล์คือ ในส่วนของ MD นี้จะมีการแก้ไขในทุกครั้งที่มีการเก็บข้อมูลใหม่ซึ่งแน่นอนว่าในช่วงเวลาเดียวกันนั้นต้องมีผู้ใช้งานที่ใช้งานพร้อมๆ กันหลายคนดังนั้นถ้าเราทำการรวม MD ของแต่ละคนเข้าเป็นไฟล์เดียวกันก็จะเกิดปัญหาในการอัปเดตส่วน MD ที่จะใช้งานซ้อนกัน

โดยอัลกอริทึมที่ใช้ในการสร้าง MD คืออัลกอริทึม MD5 มีความยาว 128 บิต

ส่วนเก็บข้อมูลพฤติกรรมผู้ใช้

โครงสร้างข้อมูลที่ใช้เก็บข้อมูลพฤติกรรมที่ส่วน IDS โฮสต์มีลักษณะดังนี้

```
struct userLog {
    char pathname[PATH_LEN];
    char command[MAX_COMM_LEN];
    unsigned long startTime;
};
```

ลักษณะการแยกเก็บข้อมูลนั้นผู้ใช้แต่ละคนที่มาจากแต่ละโฮสต์นั้นจะแยกตามหมายเลขไอพีแอดเดรสและหมายเลขประจำตัวผู้ใช้ที่ส่งมาพร้อมกับข้อมูล ยกตัวอย่างเช่น ข้อมูลพฤติกรรมผู้ใช้ที่มีหมายเลขประจำตัว 501 มาจากโฮสต์ที่มีหมายเลขไอพีแอดเดรสเป็น 161.246.2.201 ข้อมูลพฤติกรรมจะถูกเก็บอยู่ในไดเรกทอรีที่มีชื่อเป็น “161.246.10.21” และชื่อไฟล์ “501” โดยข้อมูลจะถูกเก็บเป็นเรคอร์ดต่อกันไปเรื่อยๆ

ส่วนการวิเคราะห์ข้อมูล

ลักษณะการวิเคราะห์ข้อมูลทำการวิเคราะห์ลักษณะการใช้งานคำสั่งโดยดูจากอาร์กิวเมนต์และลำดับการใช้งานคำสั่ง

ในส่วนการวิเคราะห์ข้อมูลจะแบ่งเป็น 2 ขั้นตอนคือ

1. ขั้นตอนเตรียมข้อมูลเพื่อการวิเคราะห์

เป็นขั้นตอนที่เตรียมข้อมูลพฤติกรรมที่รับมาให้อยู่ในรูปแบบที่เหมาะสมและง่ายต่อการวิเคราะห์โดยวิธีการคือนำข้อมูลคำสั่งที่เก็บอยู่ในฟิลด์ข้อมูล `command[MAX_COMM_LEN]` แล้วแทนที่อาร์กิวเมนต์ที่ไม่ใช่อปชัน(อาร์กิวเมนต์ที่เป็นอปชันจะขึ้นต้นด้วยตัวอักษร ‘-‘) ด้วยข้อความ “<” เหตุผลต้องเปลี่ยนแปลงข้อมูลดังกล่าวออกไปเนื่องจากว่าสิ่งที่บอกถึงพฤติกรรมของผู้ใช้งานจะเป็นอาร์กิวเมนต์ที่เป็นอปชันเสียมากกว่า ยกตัวอย่างเช่น เมื่อเราใช้คำสั่ง `emacs` ซึ่งเป็นโปรแกรมเท็กซ์เอดิเตอร์โดยเรียกจากเชลล์

```
#emacs -nw a.txt b.txt
```

ชื่อไฟล์ที่ตามหลังออปชันนั้นจะไม่แทนถึงพฤติกรรมผู้ใช้นักเพราะในแต่ละครั้งที่ผู้ใช้เรียกโปรแกรมนี้ขึ้นใช้งานก็มักจะเปลี่ยนชื่อไฟล์ที่เรียก แต่ออปชันยังคงเดิม ดังนั้นจากคำสั่งที่ผู้ใช้งานป้อนเข้ามาเมื่อผ่านส่วนขั้นตอนการเตรียมข้อมูลเพื่อการวิเคราะห์แล้วก็จะได้เป็นข้อความดังนี้

```
emacs -nw <f> <f>
```

แต่การเตรียมข้อมูลด้วยวิธีนี้ยังมีจุดอ่อนคือ มีบางคำสั่งที่ออปชันการใช้งานนั้นอาจไม่ได้ขึ้นต้นด้วยตัวอักษร '-' เช่น คำสั่ง tar เป็นต้น

```
#tar xvfz global-4.2.tar.gz
```

เมื่อผ่านขั้นตอนการเตรียมข้อมูลเพื่อการวิเคราะห์ผลลัพธ์ที่ได้คือ

```
tar <f> <f> <f>
```

เห็นได้ว่าออปชัน 'xvfz' นั้นหายไปด้วยเพราะไม่ได้ขึ้นต้นด้วยตัวอักษร '-' แต่อย่างไรก็ตาม คำสั่งที่ใช้ออปชันที่ไม่ได้ขึ้นต้นด้วยตัวอักษร '-' มีอยู่เป็นส่วนน้อย

2. ขั้นตอนการวิเคราะห์ข้อมูล

วิธีการที่ใช้วัดค่าความเหมือนจะใช้การวัดค่าความเหมือน(similarity measure) ของข้อมูลโดยค่าค่านี้จะมากเมื่อข้อมูลคู่ที่นำมาเปรียบเทียบนั้นมีความคล้ายกันมาก และมีค่าน้อยลงเมื่อข้อมูลคู่ต่างๆ มีความต่างกันออกไปโดยค่าที่ยิ่งมากก็ยิ่งแสดงว่าคล้ายกันมากและในทางกลับกันถ้าข้อมูลมีความต่างกันมากก็จะมีค่าน้อยลง

โดยอัลกอริทึมความเหมือนที่ใช้มีขั้นตอนดังนี้

ให้ Seq_1, Seq_2 แทนลำดับข้อมูลชุดที่ 1 และ 2 ตามลำดับ

$Seq_i(i)$, แทนข้อมูลตัวที่ i ใน Seq_i

ให้ c เป็นค่าคงที่มีค่าเริ่มต้นที่ 1 และ Sim แทนค่าความเหมือนมีค่าเริ่มต้นที่ 0

เริ่มขั้นตอนการเปรียบเทียบความเหมือนของลำดับ Seq_1, Seq_2

1. ในแต่ละตำแหน่งของ i ในลำดับที่ทำการเปรียบเทียบ

- ถ้า $Seq_1(i) = Seq_2(i)$ ให้ $Sim := Sim + c$ และเพิ่มค่าโดย $c := c + 1$
- ถ้าไม่เท่ากันให้ค่า $c = 1$

2. หลังจากทุกตำแหน่งทำการเปรียบเทียบแล้วค่า Sim ที่ได้มาคือค่าความเหมือนของลำดับทั้งสอง

คะแนนค่าความเหมือนจะมีค่าระหว่าง 0 ถึง $n*(n+1)/2$ โดย n คือความยาวของลำดับที่ทำการตรวจสอบ ยกตัวอย่างเพื่อให้เห็นภาพได้ชัดเจนดังนี้

ให้ $Seq_1 = \{ "ls", "-color", "<f>", "vi", "<f>" \}$

และ $Seq_2 = \{ "ls", "-p", "<f>", "cat", "<f>" \}$

จะได้ค่าความเหมือนของลำดับทั้งสองดังตารางโดยการเปรียบเทียบลำดับทั้งสองดูจากซ้ายไปขวา

Seq ₁		ls	- -color	<f>	vi	<f>	คะแนนหลังการเปรียบเทียบ
Seq ₂		ls	-p	<f>	cat	<f>	
C	ค่าเริ่มต้น=1	1	2	1	2	1	-
Sim	ค่าเริ่มต้น=0	0	1	1	2	2	3

ตารางที่ 4.1 แสดงการคำนวณค่าความเหมือนของลำดับ

จะได้ว่าลำดับทั้งสองนี้มีค่าความเหมือน 3

3. ค่าความเหมือนของลำดับ Seq_i ลำดับเดียวกับเซตของลำดับ L ได้ดังนี้

$$\text{Sim}(\text{Seq}_i, L) = \max_{\text{Seq}_j \in L} \{ \text{Sim}(\text{Seq}_i, \text{Seq}_j) \}$$

หมายความว่าค่าความเหมือนของลำดับ Seq_i เมื่อเปรียบเทียบกับเซตของลำดับ L แล้วคือค่า Sim ที่มีค่ามากที่สุดเมื่อทำการเปรียบเทียบ Seq_i กับทุกๆ ลำดับในเซต L

จากอัลกอริทึมด้านบนได้แสดงให้เห็นว่าในการทำการวิเคราะห์นั้นต้องมีลำดับอยู่ 2 ชุดนำมาเปรียบเทียบกัน เมื่อนำมาเปรียบเทียบกับลักษณะของฐานข้อมูลพฤติกรรมผู้ใช้งานของเราแล้วเราจะแบ่งฐานข้อมูลออกเป็น 2 ส่วนคือ

1. ฐานข้อมูลการใช้งานในช่วงระยะเวลาปัจจุบัน
2. ฐานข้อมูลการใช้งานในช่วงระยะเวลาที่ผ่านมา

แล้วนำฐานข้อมูลทั้ง 2 มาเปรียบเทียบค่าความเหมือนกันโดยในอัลกอริทึมขั้นตอนที่ 3 จะต้องนำลำดับพฤติกรรมการใช้งานในช่วงระยะเวลาปัจจุบันจำนวน 1 ลำดับมาทำการเปรียบเทียบกับฐานข้อมูลการใช้งานในช่วงระยะเวลาที่ผ่านมาทั้งหมดแล้วหาค่าสูงสุด และเมื่อได้ค่านั้นๆ ก็นำมาบวกเก็บไว้และทำการหาค่าความเหมือนของลำดับต่อไปของฐานข้อมูลการใช้งานในช่วงระยะเวลาปัจจุบันกับฐานข้อมูลการใช้งานในช่วงระยะเวลาที่ผ่านมาทั้งหมด จนกระทั่งสิ้นสุดข้อมูลของฐานข้อมูลการใช้งานในช่วงระยะเวลาปัจจุบัน ค่ารวมที่ได้คือค่าความเหมือนของพฤติกรรมการใช้งานในช่วงระยะเวลาปัจจุบันเทียบกับช่วงระยะเวลาที่ผ่านมา ซึ่งเราใช้ค่าค่านี้ในการวัดการเปลี่ยนแปลงไปของพฤติกรรมของผู้ใช้แต่ละคน หากค่านี้มีค่าสูงแสดงว่าพฤติกรรมผู้ใช้คนนั้นๆ ค่อนข้างคงที่ แต่ถ้าหากค่านี้มีค่าต่ำแสดงว่าพฤติกรรมของผู้ใช้มีการเปลี่ยนแปลงและหากมีค่าต่ำมากจนถึงระดับหนึ่งที่เราตั้งไว้ก็จะแสดงว่าพฤติกรรมผู้ใช้คนนั้นมีพิรุณ เข้าข่ายการบุกรุก

จากอัลกอริทึมข้างบนสังเกตได้ว่ามีตัวแปรที่สำคัญอยู่ 4 ตัวแปรคือ

1. จำนวนสมาชิกใน Seq แต่ละ Seq
2. จำนวนข้อมูลที่ใช้เปรียบเทียบจากฐานข้อมูลการใช้งานในช่วงระยะเวลาปัจจุบัน
3. จำนวนข้อมูลที่ใช้เปรียบเทียบจากฐานข้อมูลการใช้งานในช่วงระยะเวลาที่ผ่านมา
4. ค่าต่ำสุดที่ยอมรับได้ของการเปลี่ยนแปลงพฤติกรรม

ซึ่งค่าทั้ง 4 นี้จำเป็นต้องมีการปรับให้เหมาะสมในแต่ละระบบซึ่งจะได้อธิบายต่อไป

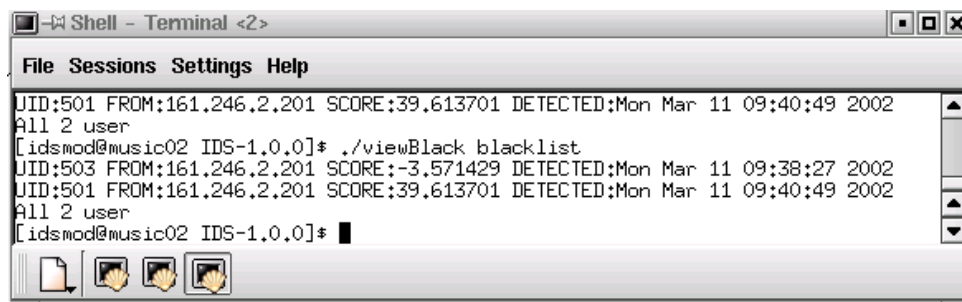
4.2.3 ส่วนตอบสนองการวิเคราะห์

เป็นส่วนที่ถูกเรียกใช้เมื่อระบบตรวจจับได้ว่ามีผู้ใช้ที่มีพฤติกรรมเปลี่ยนไปจนถึงระดับที่ผิดปกติโดยจะแบ่งการทำงานเป็น 2 ประเภทคือ

1. ทำการเก็บล็อกไว้ใน IDS โฮสต์ โดยระบบจะมีอยู่ 2 ชนิดคือ

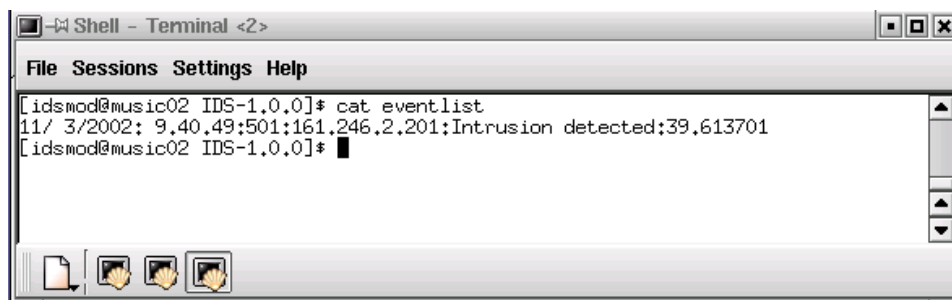
- Blacklist เก็บข้อมูลผู้ใช้งานที่เข้าข่ายมีพฤติกรรมการบุกรุกในขณะนั้นและเมื่อพฤติกรรมของผู้ใช้คนนั้นๆ เปลี่ยนแปลงไปในทางที่ดีขึ้นจนมีค่าพฤติกรรมที่เกินค่าที่กำหนดไว้ก็จะถูกลบออกจากรายชื่อนี้ โครงสร้างของไฟล์นี้จะเป็นเรคอร์ดต่อกัน โดยมีโครงสร้างดังนี้

```
struct blackUser{
    int uid;                // uid ของผู้ใช้
    char hostIP[20];        // หมายเลขไอพีแอดเดรสที่ผู้ใช้ใช้งานอยู่
    float score;            // ค่าคะแนนการบุกรุก
    unsigned long detectTime; // เวลาที่ตรวจพบการบุกรุก
}
```



รูปที่ 4.4 แสดงข้อมูลที่อยู่ในไฟล์ blacklist

- Eventlist เก็บข้อมูลเหตุการณ์ต่างๆ ทั้งหมดที่เกิดขึ้นเช่นมีผู้ใช้เริ่มมีพฤติกรรมเข้าข่ายการบุกรุก มีผู้ใช้ที่มีพฤติกรรมดีขึ้นที่เปลี่ยนสถานะจากผู้บุกรุกเป็นผู้ใช้ปกติ มีความผิดพลาดจากการตรวจสอบความถูกต้องของฐานข้อมูล เป็นต้น ไฟล์นี้มีโครงสร้างเป็นเท็กซ์ไฟล์ปกติ



The image shows a terminal window titled "Shell - Terminal <2>". The window has a menu bar with "File", "Sessions", "Settings", and "Help". The terminal content shows a user prompt "[idsmod@music02 IDS-1.0.0]#" followed by the command "cat eventlist". The output of the command is "11/ 3/2002: 9,40,49;501;161,246,2,201;Intrusion detected;39,613701". The prompt is followed by a cursor. The terminal window has a scrollbar on the right and a taskbar at the bottom with icons for a file, a folder, and a terminal.

```
[idsmod@music02 IDS-1.0.0]$ cat eventlist
11/ 3/2002: 9,40,49;501;161,246,2,201;Intrusion detected;39,613701
[idsmod@music02 IDS-1.0.0]$
```

รูปที่ 4.6 แสดงข้อมูลที่อยู่ในไฟล์ eventlist

2. แจ้งเตือนผู้ดูแลระบบ โดยการส่งข้อความเตือนไปยังผู้ดูแลระบบ